

Mikhail Razakov

Compiler Infrastructure / Language Tooling Engineer

razakov.mikhail@outlook.com
github.com/Misha1302
LinkedIn

Compiler/language-infrastructure engineer with professional LLVM and static-analysis experience. In UniversalToolchain I built dependency/conflict resolution, component/provider selection, deterministic pass ordering, typed artifact routing, and pre-execution plan checks; an architecture talk based on the project was accepted for LangDev'26.

EXPERIENCE

- 2026 — present** **ISP RAS — Static Analysis Engineer**
- Contribute to SharpChecker, ISP RAS's C#/.NET static-analysis platform.
- July — August 2026** **MCST — Compiler Engineering Intern**
- Implemented a LICM pass for LLVM 22 in C++; hoisting conditions check loop structure, memory access, side effects, and speculative safety.
 - Built a Python harness comparing program behavior before/after optimization and checking transform/no-transform cases.
- 2023–2026** **Independent / freelance engineering — product systems**
- VpnMediator: subscription, payment, and access state, device limits, Linux deployment, recovery, and rollback.
 - Diploma.school: jQuery dropdown interactions and responsive navigation; Unity licensing: server-side activation limits and Unity client integration.

PROJECTS

UniversalToolchain

Language features resolve dependencies and conflicts, select capability/runtime providers, obtain deterministic contribution ordering, and build typed artifact routes before runtime materialization.

Current manifest: 1,324/1,324 tests; dedicated checks cover interpreter/CIL parity and configuration integrity before execution.

LLVM Interprocedural Global IV Optimization

An LLVM 22 pass localizes supported induction-like globals after interprocedural effect analysis, affine evolution, and a separate applicability check; APInt models fixed-width wraparound.

29 positive/negative regression cases cover LLVM Verifier checks, structural expectations, idempotence, and observable before/after behavior.

x86-64 Compiler Backend & Register Allocation

SSA/CFG validation → liveness/interference → register allocation/spills → phi lowering → SysV x86-64 emission.

An independent assignment verifier plus native-vs-interpreter differential execution; tests cover branches, loops, spills, and parallel phi moves.

SKILLS

Compiler Infrastructure

dependency/conflict resolution, component/provider selection, deterministic pass ordering, typed artifact routing, pre-execution checks

Compiler / IR

C++23, LLVM 22/IR, CFG/SSA, loop and interprocedural analysis, transformation applicability

Backends / Runtime

.NET, interpreter/CIL parity, SysV x86-64, register allocation, runtime composition

Verification

regression suites, LLVM Verifier, interpreter/backend parity, differential execution

EDUCATION

HSE University — Software Engineering, 2026–2030

RECOGNITION

LangDev'26 — Build the Language, Then Make the Abstractions Disappear

Accepted talk on UniversalToolchain's architecture; LangDev'26 takes place October 8–9, 2026 in Málaga.

UniversalToolchain — Baltic Science and Engineering Competition

First Degree Diploma and Grand Prize "Perfection as Hope" for UniversalToolchain.

MEPhI Junior

Overall winner; First Degree Diploma, 96/100 for UniversalToolchain.

Moscow / remote